

Assignment5

February 26, 2025

```
[2]: import os
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.transforms as transforms
import torchvision.transforms.functional as TF
import torch.optim as optim
import torchtune.datasets as TTD
import numpy as np
import pandas as pd
import geopandas as gpd
import rasterio
import random
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import matplotlib.patches as mpatches
import seaborn as sns
from geotiff import GeoTiff
from torch.utils.data import Dataset, Subset, DataLoader
from sklearn.model_selection import KFold
from shapely.geometry import Point
```

ImportError: No module named 'triton'

C:\Users\edunv\AppData\Local\Programs\Python\Python312\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See

https://ipywidgets.readthedocs.io/en/stable/user_install.html

```
from .autonotebook import tqdm as notebook_tqdm
```

- Group number: 65
- Module: 5
- Emrik Dunvald 020208-5759, ADS
- Elias Samantzis 000715-6631, ADS
- Gusamanel@student.gu.se
- Gusdunvem@student.gu.se

We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part of other solutions.

- Elias hours spent: 29 hours
- Emrik hours spent: 29 hours

0.0.1 Dataset class for geotiff files

```
[3]: class GeoTIFFDataset(Dataset):
    def __init__(self, folder_path, files):
        self.folder_path = folder_path
        self.files = files

    def __len__(self):
        return len(self.files)

    def __getitem__(self, index):
        file_path = os.path.join(self.folder_path, self.files[index])

        with rasterio.open(file_path) as dataset:
            features = dataset.read(list(range(1, 6))) # feature bands
            target = dataset.read(6) # target band

        features = features.astype(np.float32) / 255.0
        features = torch.tensor(features)

        target = target.astype(np.float32)
        target = torch.tensor(target, dtype=torch.long)

        if random.random() > 0.5:
            features = TF.hflip(features)
            target = TF.hflip(target)

        if random.random() > 0.5:
            features = TF.vflip(features)
            target = TF.vflip(target)

        angle = random.choice([0, 90, 270]) # rotate both feature and target
        ↪ the same amount
        features = TF.rotate(features, angle, interpolation=TF.
        ↪ InterpolationMode.BILINEAR)
        target = torch.unsqueeze(target, 0) # add extra dimension for rotation
        target = TF.rotate(target, angle, interpolation=TF.InterpolationMode.
        ↪ NEAREST)
        target = target.squeeze(0) # remove extra layer after rotation

        return features, target
```

0.0.2 Read 5000 random geotiff files from directory 'Potsdam-Geotif'

0.0.3 Sample all files and split into 5 folds

```
[4]: files = [file for file in os.listdir('Potsdam-GeoTif') if file.endswith(".tif")]
random_sample_files = random.sample(files, min(5000, len(files)))

dataset = GeoTIFFDataset('Potsdam-GeoTif', random_sample_files)

kf = KFold(n_splits=5)

splits = list(kf.split(range(len(dataset))))

# create dataset folds
train_dataset_1 = Subset(dataset, splits[0][1])
train_dataset_2 = Subset(dataset, splits[1][1])
train_dataset_3 = Subset(dataset, splits[2][1])
train_dataset_concat = TTD.ConcatDataset([train_dataset_1, train_dataset_2,
↳ train_dataset_3])
val_dataset = Subset(dataset, splits[3][1])
test_dataset = Subset(dataset, splits[4][1])

# create dataLoaders
train_loader = DataLoader(train_dataset_concat, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
print(train_loader)
```

<torch.utils.data.dataloader.DataLoader object at 0x000002C05CB531A0>

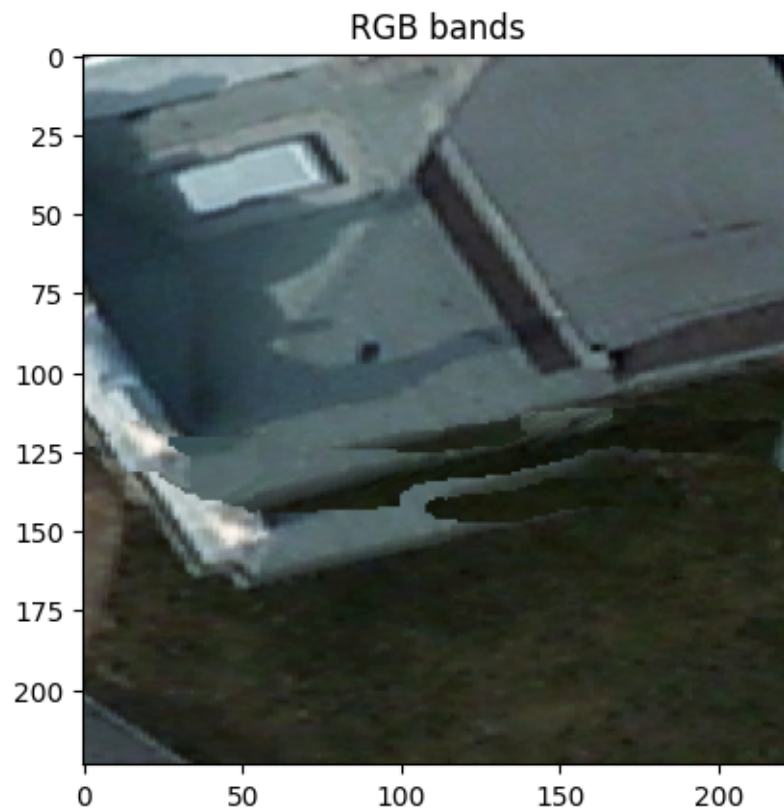
```
[5]: with rasterio.open('Potsdam-GeoTif/0000016576-0000043232.tif') as dataset:
    red = dataset.read(1)
    green = dataset.read(2)
    blue = dataset.read(3)
    elevation = dataset.read(5)
    target = dataset.read(6)

# normalize colors
red = (red - np.min(red)) / (np.max(red) - np.min(red))
green = (green - np.min(green)) / (np.max(green) - np.min(green))
blue = (blue - np.min(blue)) / (np.max(blue) - np.min(blue))

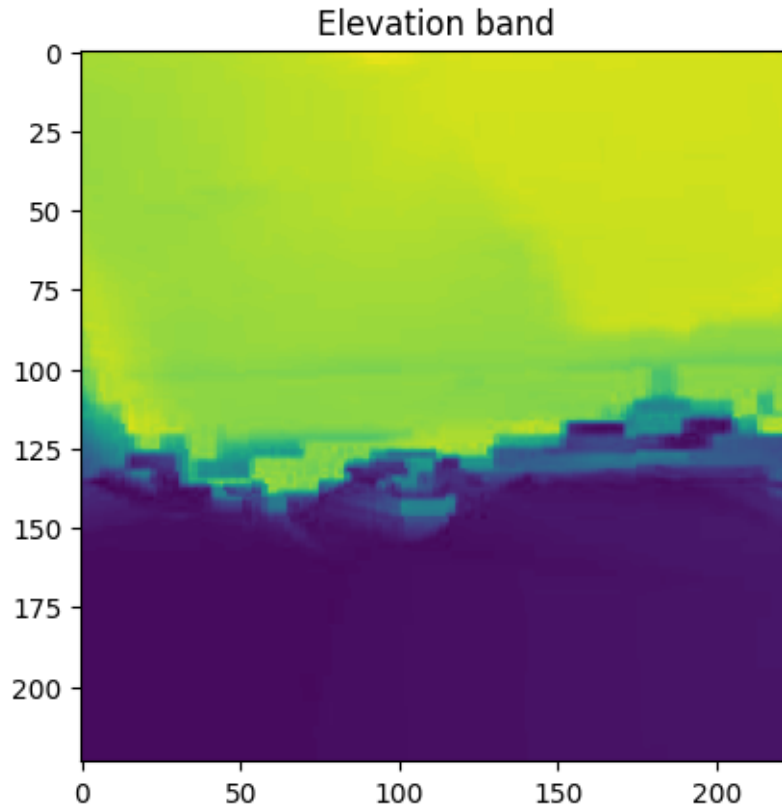
# stack bands to create an RGB image
rgb_image = np.dstack((red, green, blue))

plt.imshow(rgb_image)
plt.title("RGB bands")
```

```
plt.show()
```



```
[6]: plt.imshow(elevation)
plt.title("Elevation band")
plt.show()
```



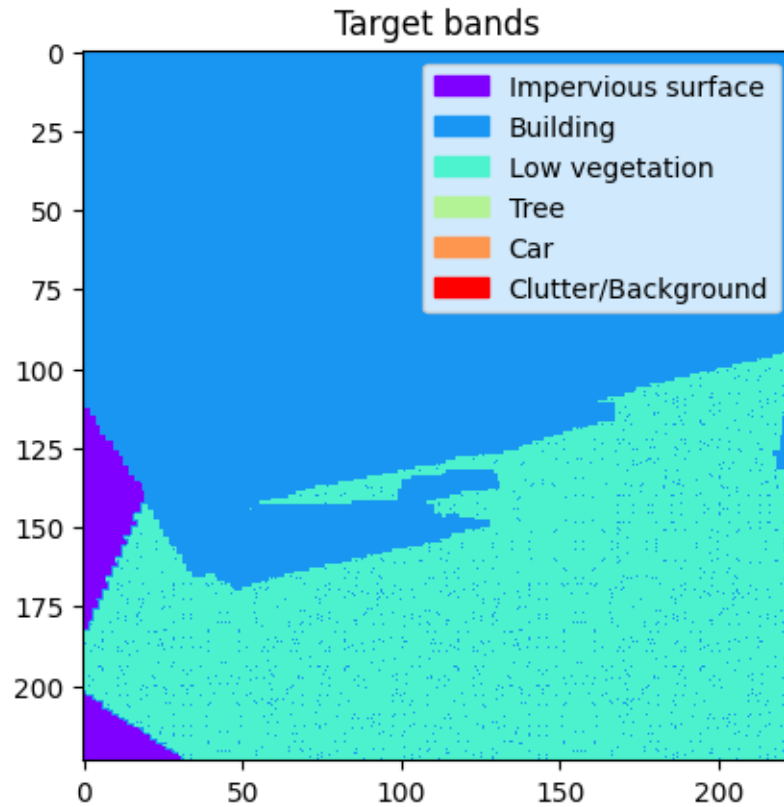
```
[7]: class_labels = {0: "Impervious surface",
                    1: "Building",
                    2: "Low vegetation",
                    3: "Tree",
                    4: "Car",
                    5: "Clutter/Background"}

# create a colormap for the classes and define color boundaries
colors = plt.get_cmap("rainbow", 6)
cmap = mcolors.ListedColormap([colors(i) for i in range(6)])
bounds = list(class_labels.keys()) + [max(class_labels.keys()) + 1]
norm = mcolors.BoundaryNorm(bounds, cmap.N)

# plot target band
im = plt.imshow(target, cmap=cmap, norm=norm)

# add labels
patches = []
for label in class_labels:
    patches.append(mpatches.Patch(color=colors(label),
    label=class_labels[label]))
```

```
plt.legend(handles=patches)
plt.title("Target bands")
plt.show()
```



0.0.4 Part 1 model with 4 bands

```
[8]: class Part1Model(nn.Module):
    def __init__(self):
        super(Part1Model, self).__init__()
        self.conv1 = nn.Conv2d(4, 32, 3, padding=1) # 4 channels in
        self.conv2 = nn.Conv2d(32, 6, 3, padding=1) # 6 channels out

        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.softmax(self.conv2(x))
        return x
```

0.0.5 Train model

```
[11]: model = Part1Model()
optimizer = optim.Adam(model.parameters())
criterion = nn.CrossEntropyLoss() # set loss to binary Cross-Entropy

# use gpu if it is available
if torch.cuda.is_available():
    device = torch.device("cuda")
    model.to(device)
    print('using gpu')
else:
    device = torch.device("cpu")
    model.to(device)

val_loss_plot = []
train_loss_plot = []
accuracy_plot = []

# Keep track of best accuracy performance on the validation set
best_train_accuracy = 0

for epoch in range(20):
    model.train()
    total_loss = 0
    for X_batch, y_batch in train_loader:
        images, targets = X_batch[:, :4, :, :].to(device), y_batch.to(device) ␣
        ↪ # Select only first 4 bands
        optimizer.zero_grad()
        outputs = model(images).squeeze()
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    # validation on val-set
    model.eval()
    val_loss = 0
    total_train = 0
    correct_train = 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            images, targets = X_batch[:, :4, :, :].to(device), y_batch.
            ↪ to(device) # Select only first 4 bands
            outputs = model(images).squeeze()
            predicted = outputs.argmax(dim=1)
```

```

        total_train += targets.nelement()
        correct_train += predicted.eq(targets).sum().item()
        val_loss += criterion(outputs, targets).item()

    train_accuracy = correct_train / total_train
    val_loss = val_loss / len(val_loader)
    val_loss_plot.append(val_loss)
    accuracy_plot.append(train_accuracy)
    train_loss_plot.append(total_loss/len(train_loader))

    # Save the model if the performance was better
    if train_accuracy > best_train_accuracy:
        best_train_accuracy = train_accuracy
        torch.save(model.state_dict(), "GeoTiffClassifier.pth")
        print("Model saved on epoch " + str(epoch))
    print("Epoch: " + str(epoch), "Train Loss: " + str(total_loss/
↪len(train_loader)), "Val Loss: " + str(val_loss), "Accuracy: " +
↪str(train_accuracy))

```

using gpu

Model saved on epoch 0

Epoch: 0 Train Loss: 1.693758690610845 Val Loss: 1.6308245249092579 Accuracy:
0.389915357940051

Model saved on epoch 1

Epoch: 1 Train Loss: 1.603515291467626 Val Loss: 1.5921646282076836 Accuracy:
0.40279536033163266

Model saved on epoch 2

Epoch: 2 Train Loss: 1.574707163141129 Val Loss: 1.5700058303773403 Accuracy:
0.5053352200255102

Model saved on epoch 3

Epoch: 3 Train Loss: 1.5545780481176172 Val Loss: 1.5457583330571651 Accuracy:
0.5396231465242347

Model saved on epoch 4

Epoch: 4 Train Loss: 1.5366823039156325 Val Loss: 1.5346882715821266 Accuracy:
0.543276686065051

Epoch: 5 Train Loss: 1.524794910816436 Val Loss: 1.5237882882356644 Accuracy:
0.5286921237244898

Model saved on epoch 6

Epoch: 6 Train Loss: 1.5137968076036332 Val Loss: 1.5164802595973015 Accuracy:
0.5474312420280613

Model saved on epoch 7

Epoch: 7 Train Loss: 1.5072417728444363 Val Loss: 1.507004275918007 Accuracy:
0.5496915258290817

Model saved on epoch 8

Epoch: 8 Train Loss: 1.5014792414421732 Val Loss: 1.5026737675070763 Accuracy:
0.5503846460459184

Epoch: 9 Train Loss: 1.4973300378373329 Val Loss: 1.5010960325598717 Accuracy:
0.5485625

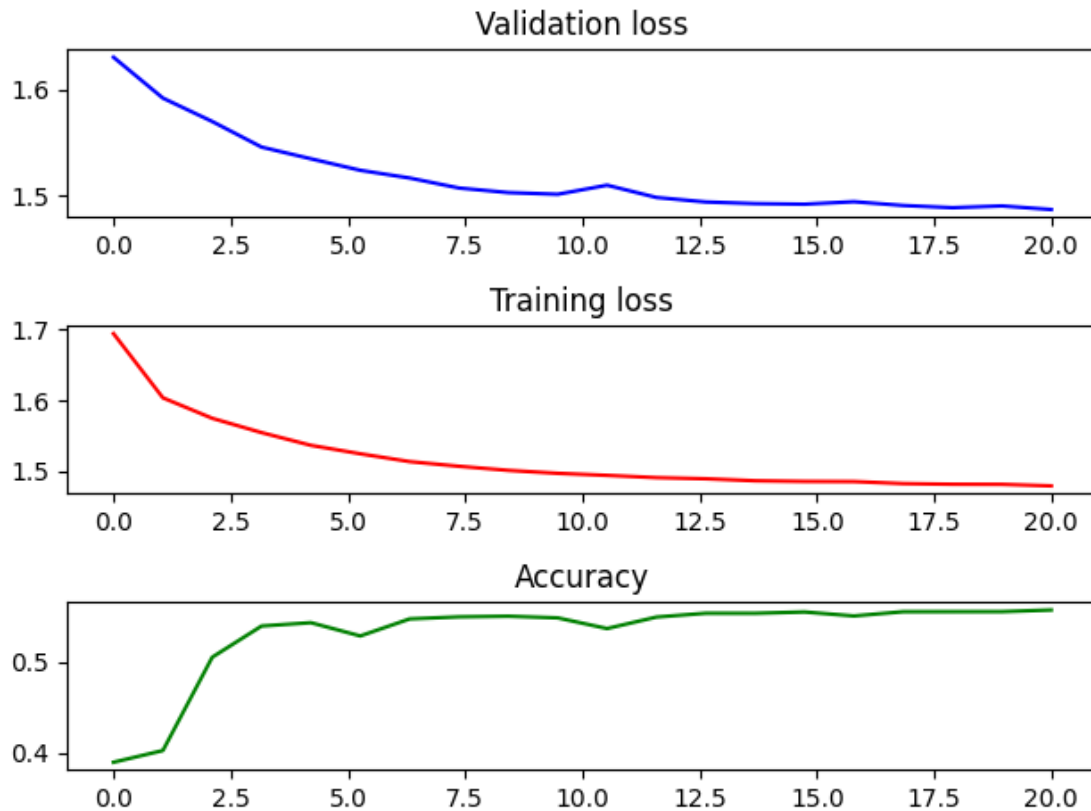
Epoch: 10 Train Loss: 1.4944768104147403 Val Loss: 1.5096858814358711 Accuracy: 0.5368530373086735
Epoch: 11 Train Loss: 1.4912753206618288 Val Loss: 1.4980667755007744 Accuracy: 0.549455078125
Model saved on epoch 12
Epoch: 12 Train Loss: 1.4896833896636963 Val Loss: 1.493784360587597 Accuracy: 0.5536430365114796
Epoch: 13 Train Loss: 1.4868773057105693 Val Loss: 1.4922580048441887 Accuracy: 0.5536071428571429
Model saved on epoch 14
Epoch: 14 Train Loss: 1.4859737474867638 Val Loss: 1.4917242676019669 Accuracy: 0.5550379663584184
Epoch: 15 Train Loss: 1.4857359376359494 Val Loss: 1.4940336272120476 Accuracy: 0.5506812818877551
Model saved on epoch 16
Epoch: 16 Train Loss: 1.4828848914897188 Val Loss: 1.4904740825295448 Accuracy: 0.555332051179847
Model saved on epoch 17
Epoch: 17 Train Loss: 1.4819907507997878 Val Loss: 1.4883735626935959 Accuracy: 0.5553780891262755
Model saved on epoch 18
Epoch: 18 Train Loss: 1.4818435463499515 Val Loss: 1.4900398403406143 Accuracy: 0.5554168327487244
Model saved on epoch 19
Epoch: 19 Train Loss: 1.4796734021065083 Val Loss: 1.4866448491811752 Accuracy: 0.5572447385204081

0.0.6 Plot train, val and accuracy

```
[12]: fig, axes = plt.subplots(3, 1)
      x = np.linspace(0.0, 20.0, 20)

      # plot the validation loss, training loss and accuracy
      axes[0].plot(x, val_loss_plot, color='blue')
      axes[0].set_title("Validation loss")
      axes[1].plot(x, train_loss_plot, color='red')
      axes[1].set_title("Training loss")
      axes[2].plot(x, accuracy_plot, color='green')
      axes[2].set_title("Accuracy")

      plt.tight_layout()
      plt.show()
```



0.0.7 Test model

```
[13]: def model_accuracy(model, loader, bands):
    # use gpu if it is available
    if torch.cuda.is_available():
        device = torch.device("cuda")
        model.to(device)
        print('using gpu')
    else:
        device = torch.device("cpu")
        model.to(device)

    # evaluate on train set
    model.eval()
    all_preds = []
    all_labels = []
    total_train = 0
    correct_train = 0
    train_accuracy = 0
    with torch.no_grad():
```

```

        for X_batch, y_batch in loader:
            images, targets = X_batch[:, :bands, :, :].to(device), y_batch.
            ↪to(device) # Select only first 4 bands
            outputs = model(images).squeeze()
            predicted = outputs.argmax(dim=1)
            total_train += targets.nelement()
            correct_train += predicted.eq(targets).sum().item()
            train_accuracy += correct_train / total_train
        print(train_accuracy/len(loader))

model = Part1Model()
model.load_state_dict(torch.load('GeoTiffClassifier.pth'))
model_accuracy(model, test_loader, 4)

```

using gpu
0.5665804167981136

0.0.8 Final Model

```

[14]: class FinalModel(nn.Module):
    def __init__(self):
        super(FinalModel, self).__init__()

        # encode
        self.conv1 = nn.Conv2d(5, 32, 3, padding=1) # 5 in, 32 out
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 32 in, 64 out
        self.conv3 = nn.Conv2d(64, 64, 3, padding=1) # 64 in, 64 out

        # pooling
        self.pool = nn.MaxPool2d(2, stride=2)

        # decode
        self.upconv1 = nn.ConvTranspose2d(64, 64, 3, stride=2, padding=1, ↪
            ↪output_padding=1) # 64 in, 64 out
        self.upconv2 = nn.ConvTranspose2d(128, 32, 3, stride=2, padding=1, ↪
            ↪output_padding=1) # 128 in, 32 out
        self.final_conv = nn.Conv2d(64, 6, 3, padding=1) # 64 in, 6 out

    def forward(self, x):
        layer_1 = F.relu(self.conv1(x))
        layer_2 = self.pool(layer_1)
        layer_2 = F.relu(self.conv2(layer_2))
        layer_3 = self.pool(layer_2)
        layer_3 = F.relu(self.conv3(layer_3))

        # decoder

```

```

        layer_4 = F.relu(self.upconv1(layer_3))
        layer_4 = torch.cat((layer_4, layer_2), dim=1)

        layer_5 = F.relu(self.upconv2(layer_4))
        layer_5 = torch.cat((layer_5, layer_1), dim=1)

        output = F.softmax(self.final_conv(layer_5), dim=1)
        return output

model = FinalModel()
x = torch.randn(1, 5, 224, 224)
output = model(x)
print(output.shape)

```

```
torch.Size([1, 6, 224, 224])
```

```

[16]: model = FinalModel()
optimizer = optim.Adam(model.parameters())
criterion = nn.CrossEntropyLoss()

# use gpu if it is available
if torch.cuda.is_available():
    model.to(torch.device("cuda"))
    print('using gpu')
else:
    model.to(torch.device("cpu"))

print(device)

final_val_loss_plot = []
final_train_loss_plot = []
final_accuracy_plot = []

# Keep track of best accuracy performance on the validation set
best_train_accuracy = 0

for epoch in range(20):
    model.train()
    total_loss = 0
    for X_batch, y_batch in train_loader:
        images, targets = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(images).squeeze()
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

```

```

        total_loss += loss.item()

    # validation on val-set
    model.eval()
    val_loss = 0
    total_train = 0
    correct_train = 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            images, targets = X_batch.to(device), y_batch.to(device) # Select
↪only first 4 bands
            outputs = model(images).squeeze()
            predicted = outputs.argmax(dim=1)
            total_train += targets.nelement()
            correct_train += predicted.eq(targets).sum().item()
            val_loss += criterion(outputs, targets).item()

    train_accuracy = correct_train / total_train
    val_loss = val_loss / len(val_loader)
    final_val_loss_plot.append(val_loss)
    final_accuracy_plot.append(train_accuracy)
    final_train_loss_plot.append(total_loss/len(train_loader))

    # Save the model if the performance was better
    if train_accuracy > best_train_accuracy:
        best_train_accuracy = train_accuracy
        torch.save(model.state_dict(), "FinalGeoTiffClassifier.pth")
        print("Model saved on epoch" + str(epoch))
    print("Epoch: " + str(epoch), "Train Loss: " + str(total_loss/
↪len(train_loader)), "Val Loss: " + str(val_loss), "Accuracy: " +
↪str(train_accuracy))

```

using gpu

cuda

Model saved on epoch0

Epoch: 0 Train Loss: 1.543455036396676 Val Loss: 1.4118370302021503 Accuracy:
0.6313314931441326

Model saved on epoch1

Epoch: 1 Train Loss: 1.4005862487123368 Val Loss: 1.3967183902859688 Accuracy:
0.6460491868622449

Model saved on epoch2

Epoch: 2 Train Loss: 1.3880012707507356 Val Loss: 1.3925650119781494 Accuracy:
0.6481330715880103

Model saved on epoch3

Epoch: 3 Train Loss: 1.3690776038677135 Val Loss: 1.3580959998071194 Accuracy:
0.6840786232461735

Model saved on epoch4

Epoch: 4 Train Loss: 1.355990062368677 Val Loss: 1.3530450724065304 Accuracy:

```

0.688798509247449
Model saved on epoch5
Epoch: 5 Train Loss: 1.3464758370785004 Val Loss: 1.343459352850914 Accuracy:
0.6984564931441326
Model saved on epoch6
Epoch: 6 Train Loss: 1.3339695626116814 Val Loss: 1.339806903153658 Accuracy:
0.702295579559949
Epoch: 7 Train Loss: 1.3317453747100019 Val Loss: 1.3550016582012177 Accuracy:
0.6850301737882653
Model saved on epoch8
Epoch: 8 Train Loss: 1.3347323752464133 Val Loss: 1.3387326262891293 Accuracy:
0.7033396045918368
Model saved on epoch9
Epoch: 9 Train Loss: 1.3291152538137232 Val Loss: 1.330074429512024 Accuracy:
0.7115535315688776
Epoch: 10 Train Loss: 1.33078847920641 Val Loss: 1.3313972651958466 Accuracy:
0.710221022002551
Model saved on epoch11
Epoch: 11 Train Loss: 1.3284697050743914 Val Loss: 1.3270877078175545 Accuracy:
0.7142701291454082
Model saved on epoch12
Epoch: 12 Train Loss: 1.3287124253333884 Val Loss: 1.3266315422952175 Accuracy:
0.714598313934949
Model saved on epoch13
Epoch: 13 Train Loss: 1.3256226968257985 Val Loss: 1.325339186936617 Accuracy:
0.7157507374043367
Epoch: 14 Train Loss: 1.3262371370132933 Val Loss: 1.3267547115683556 Accuracy:
0.7140796197385204
Epoch: 15 Train Loss: 1.3246592169112348 Val Loss: 1.3282623812556267 Accuracy:
0.7137858737244898
Epoch: 16 Train Loss: 1.3238514636425263 Val Loss: 1.325933013111353 Accuracy:
0.7152637117346938
Epoch: 17 Train Loss: 1.323325202820149 Val Loss: 1.3310286439955235 Accuracy:
0.7106504902742347
Epoch: 18 Train Loss: 1.3236436919963106 Val Loss: 1.3269228488206863 Accuracy:
0.7147870296556122
Epoch: 19 Train Loss: 1.321755510695437 Val Loss: 1.3423662558197975 Accuracy:
0.6983723692602041

```

0.0.9 Plot train, val and accuracy for the final model

```

[17]: fig, axes = plt.subplots(3, 1)
      x = np.linspace(0.0, 20.0, 20)

      # plot the validation loss, training loss and accuracy
      axes[0].plot(x, final_val_loss_plot, color='blue')
      axes[0].set_title("Validation loss")

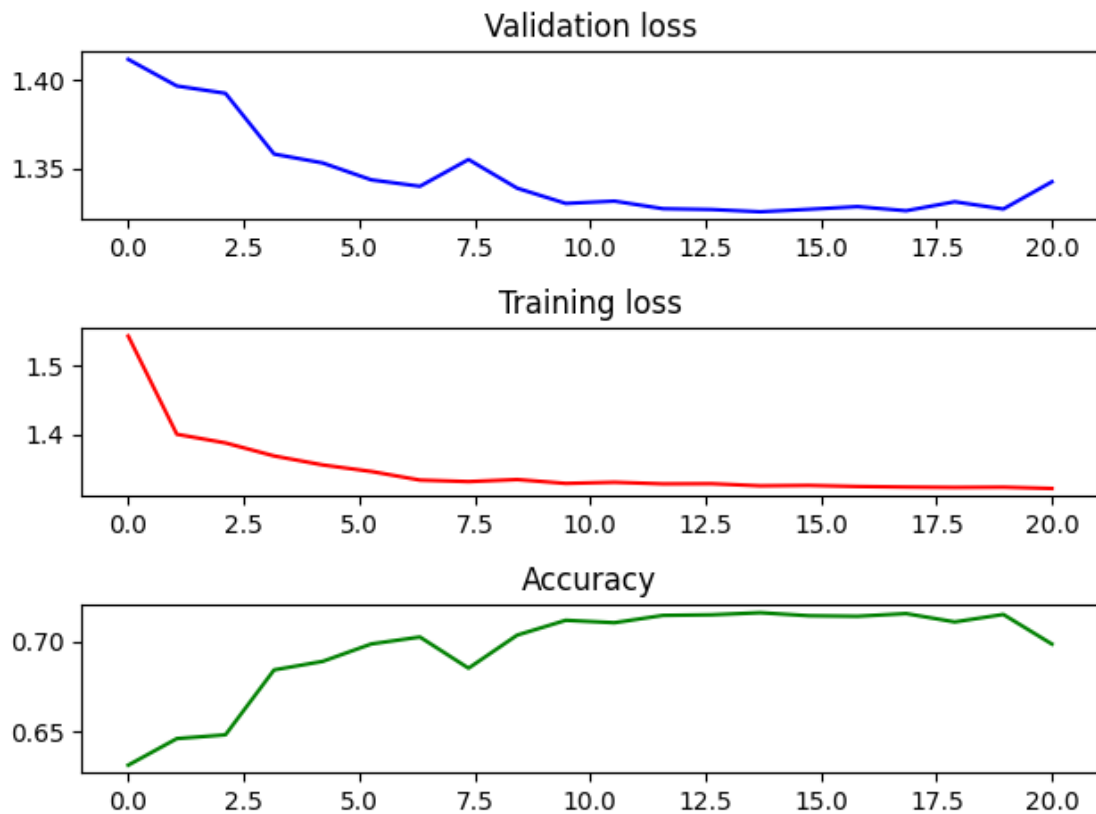
```

```

axes[1].plot(x, final_train_loss_plot, color='red')
axes[1].set_title("Training loss")
axes[2].plot(x, final_accuracy_plot, color='green')
axes[2].set_title("Accuracy")

plt.tight_layout()
plt.show()

```



0.0.10 Accuracy of final model on test set

```

[18]: model = FinalModel()
      model.load_state_dict(torch.load('FinalGeoTiffClassifier.pth'))
      model_accuracy(model, test_loader, 5)

```

```

using gpu
0.7165465304200607

```

0.0.11 Visualize a prediction

```
[19]: model = FinalModel()
model.load_state_dict(torch.load("FinalGeoTiffClassifier.pth"))
model.eval()

fig, axes = plt.subplots(2, 2, figsize=(16, 16)) # 2 rows, 2 column

# read all the bands except ir
with rasterio.open('Potsdam-GeoTif/0000016576-0000043232.tif') as dataset:
    red = dataset.read(1)
    green = dataset.read(2)
    blue = dataset.read(3)
    elevation = dataset.read(5)
    target = dataset.read(6)
    input_image = torch.tensor(dataset.read([1, 2, 3, 4, 5]), dtype=torch.
↪float32)
    input_image = input_image.unsqueeze(0)

#-----#

# plot the rgb bands

red = (red - np.min(red)) / (np.max(red) - np.min(red))
green = (green - np.min(green)) / (np.max(green) - np.min(green))
blue = (blue - np.min(blue)) / (np.max(blue) - np.min(blue))

rgb_image = np.dstack((red, green, blue))

axes[0][0].imshow(rgb_image)
axes[0][0].set_title("RGB bands")

#-----#

# plot the model predicted labels for the pixels

with torch.no_grad():
    output = model(input_image)

predicted_class = torch.argmax(output, dim=1)
predicted_class = predicted_class.squeeze(0) # remove batch dimension

# add labels
patches = []
for label in class_labels:
```

```

    patches.append(mpatches.Patch(color=colors[label],
↪label=class_labels[label]))

# create a colormap for the classes and define color boundaries
colors = plt.get_cmap("rainbow", 6)
cmap = mcolors.ListedColormap([colors(i) for i in range(6)])
bounds = list(class_labels.keys()) + [max(class_labels.keys()) + 1] # Define
↪color boundaries
norm = mcolors.BoundaryNorm(bounds, cmap.N)

im = axes[0][1].imshow(predicted_class, cmap=cmap, norm=norm)

# create a legend with colored boxes
patches = [mpatches.Patch(color=colors(i), label=class_labels[i]) for i in
↪class_labels]

axes[0][1].legend(handles=patches)
axes[0][1].set_title("Model prediction")

#-----#

# plot the target bands

im = axes[1][0].imshow(target, cmap=cmap, norm=norm)
axes[1][0].legend(handles=patches)
axes[1][0].set_title("Target band")

#-----#

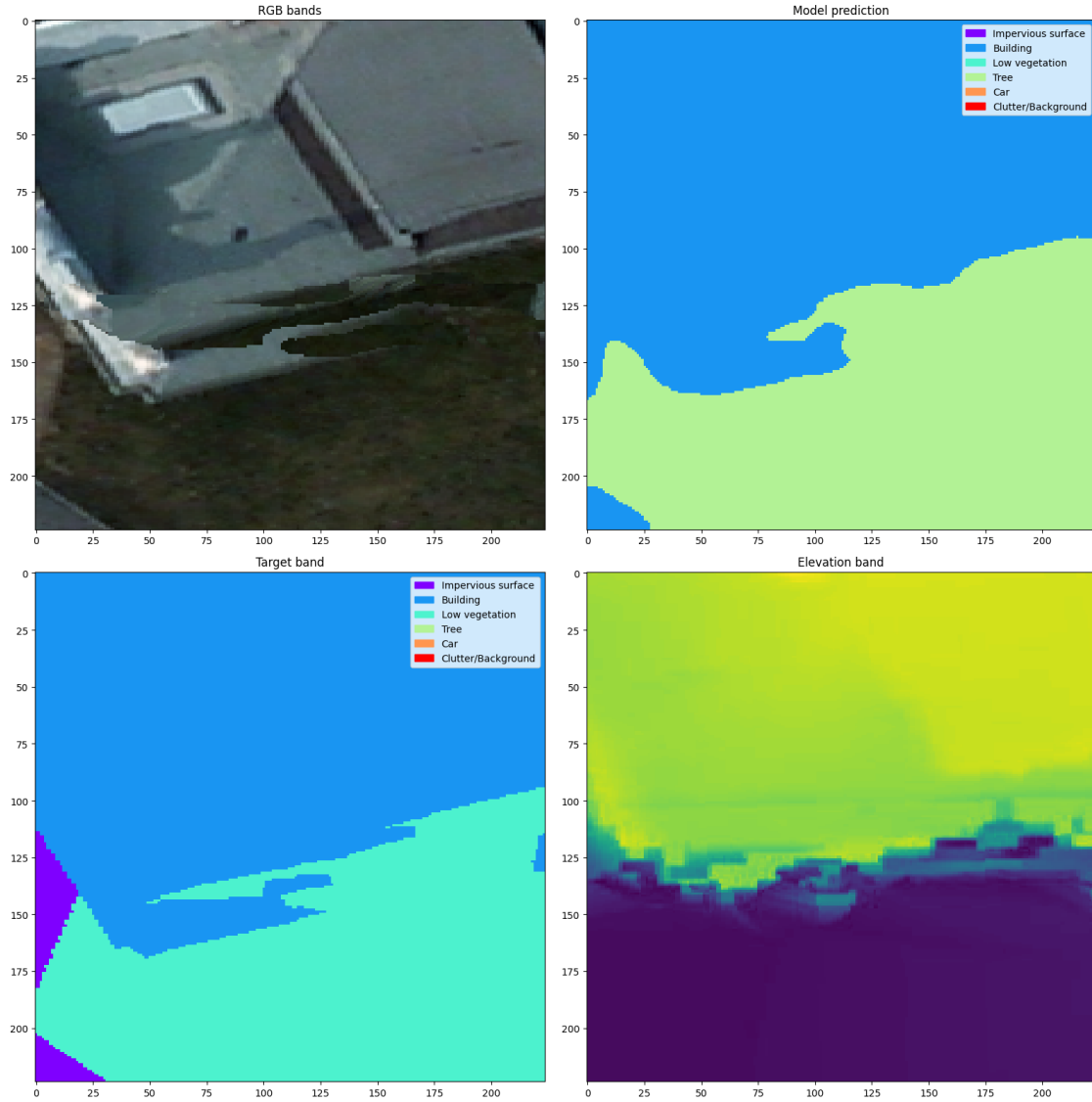
# plot the elevation bands

axes[1][1].imshow(elevation)
axes[1][1].set_title("Elevation band")

#-----#

plt.tight_layout()
plt.show()

```



1 Part 4

```
[20]: def spatial_kfold_partition(image_paths, k=5, delta=None):
    spatial_data = []

    for img_path in image_paths:
        path = os.path.join('Potsdam-GeoTif', img_path)
        with rasterio.open(path) as src:
            center_x = (src.bounds.left + src.bounds.right) / 2
            center_y = (src.bounds.top + src.bounds.bottom) / 2
            spatial_data.append((img_path, center_x, center_y))
```

```

df = pd.DataFrame(spatial_data, columns=['path', 'longitude', 'latitude'])
geometry = []
for xy in zip(df['longitude'], df['latitude']):
    geometry.append(Point(xy))
gdf = gpd.GeoDataFrame(df, geometry=geometry)

if delta is None:
    delta = (gdf['longitude'].max() - gdf['longitude'].min()) / k # adjust
    ↪ delta based on the spread

# partition as spatial grid
gdf['grid_x'] = (gdf['longitude'] // delta).astype(int)
gdf['grid_y'] = (gdf['latitude'] // delta).astype(int)

# each region is assigned to a fold after the process is finished
gdf['fold'] = (gdf['grid_x'] + gdf['grid_y']) % k

return gdf

```

```

[21]: geotiff_files = [f for f in os.listdir('Potsdam-GeoTif') if f.endswith('.tif')]

sampled_files = random.sample(geotiff_files, min(5000, len(geotiff_files)))

gdf = spatial_kfold_partition(sampled_files, k=5, delta=None)

# save
gdf[['path', 'longitude', 'latitude', 'fold']].to_csv("spatial_folds.csv",
    ↪ index=False)

```

```

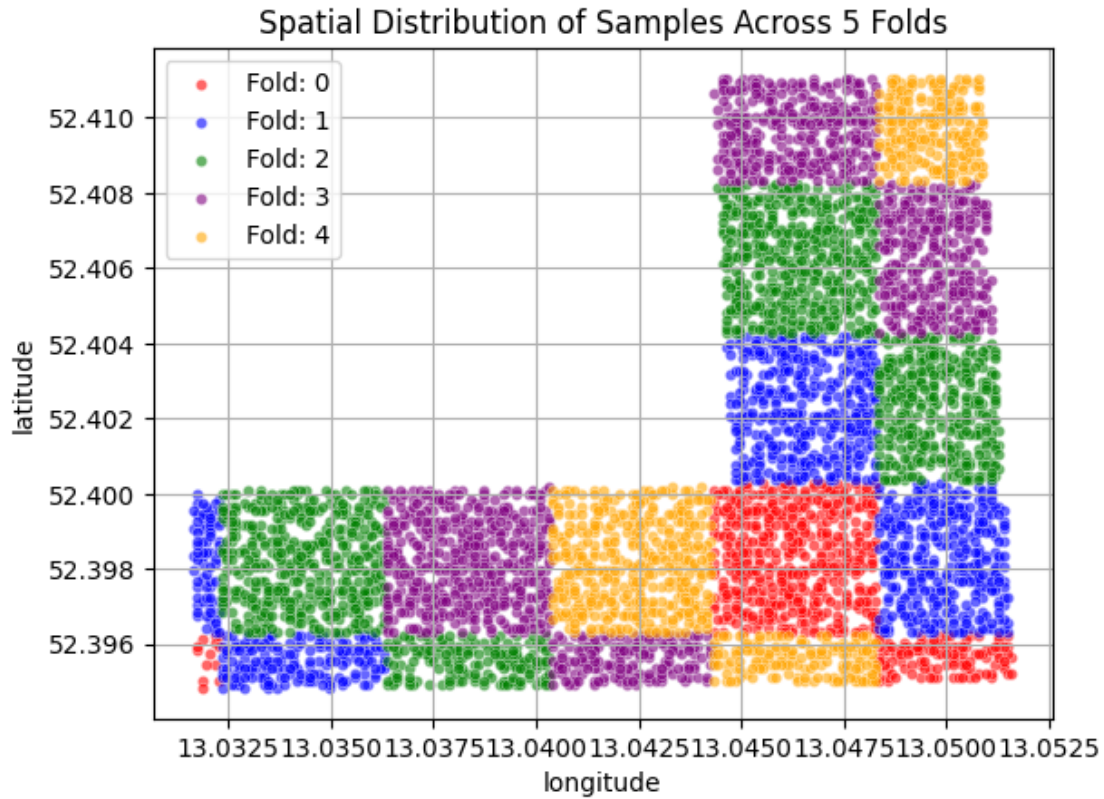
[22]: fold_colors = ['red', 'blue', 'green', 'purple', 'orange']

# loop through each fold and plot it with a color
for fold_id in range(gdf['fold'].nunique()):
    fold_data = gdf[gdf["fold"] == fold_id]
    sns.scatterplot(
        x=fold_data["longitude"],
        y=fold_data["latitude"],
        color=fold_colors[fold_id % len(fold_colors)],
        label="Fold: " + str(fold_id), alpha=0.6, s=20
    )

plt.title("Spatial Distribution of Samples Across 5 Folds")
plt.legend()
plt.grid(True)

plt.show()

```



1.0.1 Create datasets based on the new splits

```
[23]: df = pd.read_csv('spatial_folds.csv')

fold_1 = df.loc[df['fold']==0, 'path'].reset_index(drop=True)
fold_2 = df.loc[df['fold']==1, 'path'].reset_index(drop=True)
fold_3 = df.loc[df['fold']==2, 'path'].reset_index(drop=True)
fold_4 = df.loc[df['fold']==3, 'path'].reset_index(drop=True)
fold_5 = df.loc[df['fold']==4, 'path'].reset_index(drop=True)

dataset_1 = GeoTIFFDataset('Potsdam-GeoTif', fold_1)
dataset_2 = GeoTIFFDataset('Potsdam-GeoTif', fold_2)
dataset_3 = GeoTIFFDataset('Potsdam-GeoTif', fold_3)
dataset_4 = GeoTIFFDataset('Potsdam-GeoTif', fold_4)
dataset_5 = GeoTIFFDataset('Potsdam-GeoTif', fold_5)

part4_train_dataset_concat = TTD.ConcatDataset([dataset_1, dataset_2,
↪dataset_3])

part4_train_loader = DataLoader(part4_train_dataset_concat, batch_size=32,
↪shuffle=True)
```

```
part4_val_loader = DataLoader(dataset_4, batch_size=32, shuffle=True)
part4_test_loader = DataLoader(dataset_5, batch_size=32, shuffle=True)
```

1.0.2 Train classifier on new split

```
[25]: model = FinalModel()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())

# use gpu if it is available
if torch.cuda.is_available():
    model.to(torch.device("cuda"))
    print('using gpu')
else:
    model.to(torch.device("cpu"))

print(device)

part4_val_loss_plot = []
part4_train_loss_plot = []
part4_accuracy_plot = []

# Keep track of best accuracy performance on the validation set
best_train_accuracy = 0

for epoch in range(20):
    model.train()
    total_loss = 0
    for X_batch, y_batch in part4_train_loader:
        images, targets = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(images).squeeze()
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    # Validation step
    model.eval()
    val_loss = 0
    total_train = 0
    correct_train = 0
    with torch.no_grad():
        for X_batch, y_batch in part4_val_loader:
            images, targets = X_batch.to(device), y_batch.to(device) # Select
            → only first 4 bands
            outputs = model(images).squeeze()
```

```

        predicted = outputs.argmax(dim=1)
        total_train += targets.nelement()
        correct_train += predicted.eq(targets).sum().item()
        val_loss += criterion(outputs, targets).item()

    train_accuracy = correct_train / total_train
    val_loss = val_loss / len(part4_val_loader)
    part4_val_loss_plot.append(val_loss)
    part4_accuracy_plot.append(train_accuracy)
    part4_train_loss_plot.append(total_loss/len(part4_train_loader))

    # Save the model if the performance was better
    if train_accuracy > best_train_accuracy:
        best_train_accuracy = train_accuracy
        torch.save(model.state_dict(), "Part4GeoTiffClassifier.pth")
        print("Model saved on epoch" + str(epoch))
    print("Epoch: " + str(epoch), "Train Loss: " + str(total_loss/
↪len(part4_train_loader)), "Val Loss: " + str(val_loss), "Accuracy: " +
↪str(train_accuracy))

```

using gpu

cuda

Model saved on epoch0

Epoch: 0 Train Loss: 1.5344290086563597 Val Loss: 1.3958055759087586 Accuracy:
0.6438960061226537

Model saved on epoch1

Epoch: 1 Train Loss: 1.3515762103364823 Val Loss: 1.3471513375257835 Accuracy:
0.6964182226236201

Model saved on epoch2

Epoch: 2 Train Loss: 1.3314752528007994 Val Loss: 1.3324375305420313 Accuracy:
0.7099787719938497

Model saved on epoch3

Epoch: 3 Train Loss: 1.319638968782222 Val Loss: 1.3222752106495392 Accuracy:
0.7203844073995971

Model saved on epoch4

Epoch: 4 Train Loss: 1.3123737358032388 Val Loss: 1.3204273107724311 Accuracy:
0.7218691650014741

Model saved on epoch5

Epoch: 5 Train Loss: 1.3113436635504379 Val Loss: 1.317132219290122 Accuracy:
0.7258371879299472

Epoch: 6 Train Loss: 1.3082255467455437 Val Loss: 1.3296978045732548 Accuracy:
0.7132565280325613

Epoch: 7 Train Loss: 1.3098575815241387 Val Loss: 1.3259954758179493 Accuracy:
0.7166885084566531

Model saved on epoch8

Epoch: 8 Train Loss: 1.310693400971433 Val Loss: 1.3129643171261518 Accuracy:
0.7292561323787139

Epoch: 9 Train Loss: 1.3084782767803111 Val Loss: 1.3144357387836163 Accuracy:

```

0.7280017836413257
Epoch: 10 Train Loss: 1.305041270053133 Val Loss: 1.345077789746798 Accuracy:
0.6953981375606021
Epoch: 11 Train Loss: 1.3132133217568094 Val Loss: 1.3149553262270415 Accuracy:
0.7269828822128771
Model saved on epoch12
Epoch: 12 Train Loss: 1.3082265739745282 Val Loss: 1.308964723195785 Accuracy:
0.7334628496136617
Epoch: 13 Train Loss: 1.30263400458275 Val Loss: 1.309227277071048 Accuracy:
0.733090100694672
Epoch: 14 Train Loss: 1.3013930130512157 Val Loss: 1.310647258391747 Accuracy:
0.7316003526335293
Model saved on epoch15
Epoch: 15 Train Loss: 1.2989926642559944 Val Loss: 1.3060087057260366 Accuracy:
0.7358833183278327
Epoch: 16 Train Loss: 1.3017776760649173 Val Loss: 1.319957577265226 Accuracy:
0.7224818078565696
Epoch: 17 Train Loss: 1.2986796523662323 Val Loss: 1.3123827805885901 Accuracy:
0.7294558307246454
Model saved on epoch18
Epoch: 18 Train Loss: 1.2973139907451385 Val Loss: 1.3042576618683643 Accuracy:
0.7380052552095276
Model saved on epoch19
Epoch: 19 Train Loss: 1.3027585567312037 Val Loss: 1.3027887069261992 Accuracy:
0.7392594759864218

```

1.0.3 Plot training, validation and accuracy for part 4

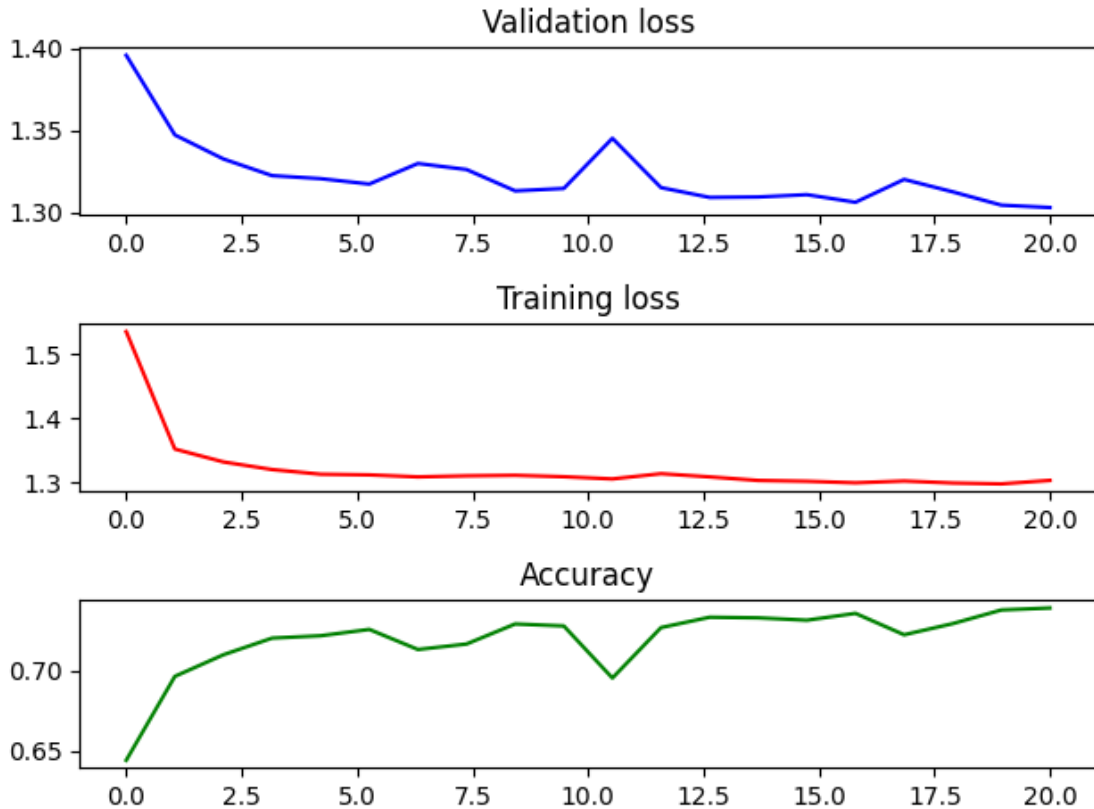
```

[26]: fig, axes = plt.subplots(3, 1)
      x = np.linspace(0.0, 20.0, 20)

      # plot the validation loss, training loss and accuracy
      axes[0].plot(x, part4_val_loss_plot, color='blue')
      axes[0].set_title("Validation loss")
      axes[1].plot(x, part4_train_loss_plot, color='red')
      axes[1].set_title("Training loss")
      axes[2].plot(x, part4_accuracy_plot, color='green')
      axes[2].set_title("Accuracy")

      plt.tight_layout()
      plt.show()

```



1.0.4 Accuracy of final model on the new split

```
[27]: model = FinalModel()
      model.load_state_dict(torch.load('Part4GeoTiffClassifier.pth'))
      model_accuracy(model, part4_test_loader, 5)
```

```
using gpu
0.6745629022578498
```

2 Description of our new split-test method

Our new train/test split method is based on a spatial k-fold partitioning approach. This means that instead of randomly splitting the dataset, we divide the study area into grid-based sections and assign each section to a specific fold. The goal is to ensure that training and testing data come from separate geographic regions, preventing data leakage and improving the models ability to generalize to unseen locations.

To do this we first extract the latitude and longitude of each data sample from the TIFF images. Then, we use a checkerboard-like partitioning method where each sample is assigned to a fold based on its location in a spatial grid. The size of the grid is controlled by a parameter called delta, which defines the spacing of the partitions. The dataset is split into k folds, and each fold is assigned a

number in a repeating pattern across the study area.

During model training we use one fold as the test set, while the remaining folds are used as the training set. This process is repeated for each fold basically, cross-validation which allows the model to be evaluated multiple times in different geographic locations. This method ensures that our model does not just memorize patterns from one region but learns to make accurate predictions in new locations.

By using this spatial aware splitting strategy, we make sure that the model is tested in areas it has never seen before.

3 Is spatial k-folds partitioning usefull for our data?

Checkerboard partitioning divides data into squares, like a checkerboard, to separate training and test data more clearly. With this method, our test samples are likely farther in geographic distance from our training samples, basically making the model predictions a more challenging task because the model cannot rely on spatial autocorrelation in the data. Our model was trained on three different spatial folds, while it was validated on one and tested on the last remaining fold. This also influenced the accuracy as our model was forced to make predictions on unseen data. This would explain why we got a 65% accuracy on the test set after implementing the spatial cross-validation method, compared to our 73% accuracy when using the normal split-test method. This suggests that we previously had data leakage which might be one reason to use this type of partition for our data.

Training a model using spatial cross-validation ensures that the model is tested on truly unseen data and, therefore, accurately evaluates how well the model generalizes to new geographic areas. Since spatial cross-validation explicitly separates training and test sets geographically, it significantly reduces the risk of spatial data leakage. This sharper separation between training and test sets helps prevent situations where information from the test set accidentally finds itself used during the training process, which leads to a more realistic and reliable assessment of the model's predictive ability. Since the model is trained on a geographically distinct area, spatial cross-validation is a valid method to test how well the model performs when predicting areas not covered during the training process.

One other thing we noticed when looking at the data for this assignment was that many of the images were quite distinct and not very similar. Looking at the whole dataset in the provided link we see that the images together are from a larger city where we couldn't see much of a difference between the different regions. This lead us to the conclusion that spatial partitioning might not be very usefull in our case since we don't have distinct regions on the map which would require it. The advantage of using this type of partition is that we can train on data from geographically distinct regions, but that would make more sense if we had a bigger more diverse map. We do however still see that there are some advantages to using this type of partition since we are getting a more accurate assesment of how good the model is when it comes to making predictions about new regions. If we wanted to improve our model it would be more appropriate to use this method since we would have a more general and robust model.

4 Summary

In Rolf et al.'s paper, the method used to split their data set into training and testing parts is called checkerboard partitioning, where this method divides the geographic areas into squares arranged like a checkerboard. Rolf and colleagues trained their ML model on data from one set of squares (black squares) and then tested the model on data from the remaining squares (white squares). They chose this approach because it clearly separates the locations of the training data from the test data. By doing this, the test samples become geographically farther away from the training samples, therefore minimizing data leakage. This method is important because it makes predicting more challenging as the model is forced to make predictions on completely unseen data. When training and testing samples are close together, models can often predict accurately by using similarities between nearby areas, basically the model utilizes the autocorrelation that can be found between these samples. However, when samples are far apart the model can't simply rely on similarities from nearby locations, making predictions more challenging but also more realistic. The reason Rolf et al. selected checkerboard partitioning is to see how well their model can predict outcomes in areas that are geographically distant from the areas it learned from. This is important because, in real-world situations we often need to predict conditions in places without direct data. Using checkerboard partitioning helps researchers understand if their method can truly generalize to new areas rather than simply memorizing patterns from similar nearby places.